

Informe técnico integral

Backend con Django RF y pasarelas de pago (Stripe, PayPal, Redsys)

Sumario

1. ¿Qué es este proyecto?.....	2
2. Tecnologías utilizadas y por qué.....	2
3. Instalación paso a paso.....	3
4. Estructura del proyecto.....	4
5. El archivo más importante: models.py.....	5
5.1. Provider.....	5
5.2. Transaction.....	5
6. serializers.py: validación de datos.....	6
7. views.py: el corazón del proyecto.....	7
7.1. ViewSets.....	7
7.2. Integración con Stripe.....	7
7.3. Webhook de Stripe.....	7
7.4. Integración con Redsys.....	8
7.5. Integración con PayPal.....	8
8. Seguridad: el archivo .env.....	9
9. El frontend.....	10

1. ¿Qué es este proyecto?

Es una aplicación web que actúa como **gestor central de pagos**. Imagina una tienda online que quiere aceptar pagos con **Stripe**, **PayPal** y **Redsys**. En lugar de integrar cada pasarela por separado en cada tienda, este sistema centraliza todo: registra los pagos, gestiona los errores y actualiza automáticamente el estado de cada transacción.

Tiene dos partes:

- **Backend:** una API REST que gestiona los datos y se comunica con las pasarelas.
- **Frontend:** un panel web para gestionar todo visualmente.

2. Tecnologías utilizadas y por qué.

- **Python:** el lenguaje de programación. Se eligió porque es claro, muy usado en desarrollo web y el enunciado lo exigía.
- **Django:** un framework web para Python. Un framework es como un esqueleto que ya tiene resueltas las partes comunes de cualquier web (base de datos, URLs, seguridad...). Django nos ahorra escribir miles de líneas de código.
- **Django REST Framework (DRF):** una extensión de Django específica para crear APIs REST. Una API REST es una forma estándar de comunicarse entre sistemas usando HTTP (GET, POST, PUT, DELETE).
- **SQLite:** la base de datos. Es un archivo que almacena todos los datos. Se eligió por su simplicidad en desarrollo, ya que no requiere instalar ningún servidor de base de datos.
- **Stripe, PayPal, Redsys:** las tres pasarelas de pago integradas. Cada una tiene su propia forma de funcionar y su propio SDK (librería oficial).
- **Bootstrap 5:** librería CSS para hacer el frontend visualmente atractivo sin escribir CSS desde cero.

3. Instalación paso a paso.

Todo empezó clonando el repositorio y creando un entorno virtual:

```
bash

git clone https://github.com/SalvaIES/payment-gateway-api.git
cd payment-gateway-api
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

¿Qué es un entorno virtual? Es una carpeta aislada con su propia instalación de Python y librerías. Evita conflictos entre proyectos, ya que cada proyecto tiene sus propias versiones de librerías sin interferir con otros.

¿Qué es **requirements.txt**? Es un archivo que lista todas las librerías necesarias:

```
Django>=5.2
django-rest-framework
python-dotenv
requests
stripe
pyresys
paypalrestsdk
flake8
tzdata
```

4. Estructura del proyecto.

```
payment-gateway-api/  
├─ config/           ← configuración de Django  
│  ├─ settings.py   ← ajustes globales  
│  └─ urls.py       ← rutas principales  
├─ core/            ← app principal (API)  
│  ├─ models.py     ← estructura de la base de datos  
│  ├─ serializers.py ← validación de datos  
│  ├─ views.py      ← lógica de negocio y pagos  
│  └─ urls.py       ← rutas de la API  
│  └─ tests.py      ← tests automáticos  
├─ frontend/       ← app del panel web  
│  ├─ views.py      ← lógica del frontend  
│  └─ urls.py       ← rutas del frontend  
│  └─ templates/    ← archivos HTML  
├─ .env             ← claves secretas (NO en GitHub)  
├─ manage.py        ← herramienta de comandos Django  
└─ requirements.txt ← lista de librerías
```

5. El archivo más importante: `models.py`

Los modelos definen cómo se guardan los datos en la base de datos. Tenemos dos:

5.1. Provider.

Provider representa una pasarela de pago:

```
python
class Provider(models.Model):
    name = models.CharField(max_length=100)      # nombre: "Stripe", "PayPal"...
    api_key = models.CharField(max_length=255)    # clave secreta de la pasarela
    environment = models.CharField(...)          # "sandbox" o "production"
    is_active = models.BooleanField(default=True) # si está activo o no
    created_at = models.DateTimeField(auto_now_add=True) # fecha automática
```

5.2. Transaction.

Transaction representa un intento de pago.

```
python
class Transaction(models.Model):
    provider = models.ForeignKey(Provider, ...) # qué pasarela se usó
    amount = models.DecimalField(...)           # cuánto dinero
    currency = models.CharField(...)            # "EUR", "USD"...
    status = models.CharField(...)              # "pending", "completed", "failed"
    incident_type = models.CharField(...)        # "impago", "error_conexion"...
    stripe_payment_intent_id = models.CharField(...) # ID del pago en Stripe
    stripe_client_secret = models.CharField(...)   # para confirmar el pago
```

6. **serializers.py**: validación de datos.

Un **serializer** hace dos cosas: convierte objetos **Python** a **JSON** (para enviar al cliente) y valida los datos que llegan antes de guardarlos.

python

```
def validate_amount(self, value):
    """El importe debe ser mayor que cero."""
    if value <= 0:
        raise serializers.ValidationError("El importe debe ser mayor que cero.")
    return value

def validate(self, data):
    """Si el estado es failed, la incidencia es obligatoria."""
    if data.get('status') == 'failed' and not data.get('incident_type'):
        raise serializers.ValidationError("Debe especificar un tipo de incidencia.")
    return data
```

7. **views.py**: el corazón del proyecto.

Aquí está toda la lógica. Veamos las partes más importantes:

7.1. ViewSets.

ViewSet gestionan automáticamente **GET, POST, PUT, DELETE**:

```
python

class TransactionViewSet(viewsets.ModelViewSet):
    queryset = Transaction.objects.all()
    serializer_class = TransactionSerializer
    authentication_classes = [TokenAuthentication] # requiere token
    permission_classes = [IsAuthenticated] # requiere estar autenticado
```

7.2. Integración con Stripe.

Al crear una transacción **Stripe**, se crea un **PaymentIntent**:

```
python

intent = stripe.PaymentIntent.create(
    amount=int(amount * 100), # Stripe trabaja en céntimos
    currency=currency,
    automatic_payment_methods={'enabled': True, 'allow_redirects': 'never'}
)
transaction = serializer.save(
    stripe_payment_intent_id=intent.id,
    stripe_client_secret=intent.client_secret
)
```

7.3. Webhook de Stripe.

Webhook de Stripe actualiza el estado cuando **Stripe** confirma el pago:

```
python

if event['type'] == 'payment_intent.succeeded':
    Transaction.objects.filter(
        stripe_payment_intent_id=payment_intent_id
    ).update(status='completed')
```

7.4. Integración con Redsys.

Integración con Redsys genera el formulario firmado:

```
python
merchant_params = MerchantParameters(
    DS_MERCHANT_AMOUNT=int(transaction.amount * 100),
    DS_MERCHANT_ORDER=f'P{str(transaction.id).zfill(11)}',
    DS_MERCHANT_MERCHANTCODE=int(settings.REDSYS_MERCHANT_CODE),
    ...
)
signed = request_payment(settings.REDSYS_SECRET_KEY, merchant_params)
```

7.5. Integración con PayPal.

Integración con PayPal crea el pago y redirige al usuario:

```
python
payment = paypalrestsdk.Payment({
    'intent': 'sale',
    'payer': {'payment_method': 'paypal'},
    'redirect_urls': {
        'return_url': f'http://127.0.0.1:8000/api/pay/paypal/execute/?transaction_id={transaction_id}',
        'cancel_url': 'http://127.0.0.1:8000/transactions/',
    },
    'transactions': [{'amount': {'total': str(transaction.amount), 'currency': transaction.currency}}]
})
```


8. Seguridad: el archivo `.env`

Las claves secretas (**Stripe**, **PayPal**, **Redsys**) nunca se escriben directamente en el código. Se guardan en un archivo `.env` que está en `.gitignore`, por lo que nunca se sube a **GitHub**:

```
STRIPE_SECRET_KEY=sk_test_...  
PAYPAL_CLIENT_ID=AcXYVu...  
REDSYS_SECRET_KEY=sq7HjrU...
```

En `settings.py` se leen así:

```
python  
  
from dotenv import load_dotenv  
import os  
load_dotenv()  
STRIPE_SECRET_KEY = os.getenv('STRIPE_SECRET_KEY')
```

9. El frontend.

El **frontend** usa **Django Templates**: **HTML** con etiquetas especiales de **Django**. Por ejemplo, para mostrar la lista de transacciones:

```
html

{% for t in transactions %}
<tr>
  <td>{{ t.id }}</td>
  <td>{{ t.provider_name }}</td>
  <td>{{ t.amount }} {{ t.currency }}</td>
  {% if t.status == 'pending' and t.provider_name == 'Stripe' %}
  <a href="/api/pay/stripe/{{ t.id }}" class="btn btn-primary">Pagar</a>
  {% endif %}
</tr>
{% endfor %}
```

Las llaves dobles `{{ }}` muestran datos. Los bloques `{% %}` son lógica (bucles, condiciones).